Handling **high concurrency in a Spring Boot application** means designing the system so it can handle **thousands of simultaneous requests efficiently without bottlenecks**. This requires optimization across multiple layers:

- Application server (threads)
- Database
- Caching
- Asynchronous processing
- Architecture

Below is a **practical architecture used in production systems**.

---

1. Optimize the Web Server Thread Pool

Spring Boot runs on embedded servers like:

- **Tomcat (default)**
- Jetty
- Undertow

Each request uses a **thread**.

Default Tomcat thread pool

```
server.tomcat.threads.max=200
server.tomcat.threads.min-spare=10
```

Example tuning:

```
server.tomcat.threads.max=400
server.tomcat.accept-count=100
```

Meaning:

```
Incoming requests
        |
        ▼
   Request Queue
```

```
      |
      ▼
  Worker Thread Pool
```

If threads are exhausted:

```
Requests wait in queue
```

---

2. Use Connection Pooling (HikariCP)

Database connections are expensive.

Example configuration:

```
spring.datasource.hikari.maximum-pool-size=30
spring.datasource.hikari.minimum-idle=10
spring.datasource.hikari.connection-timeout=30000
```

Typical production ratio:

```
App threads : DB connections
      200    :      30
```

Threads share database connections efficiently.

---

3. Use Caching (Huge Performance Gain)

If multiple users request the same data:

Example:

```
GET /patients/123
```

Instead of hitting the database each time:

```
Request → Cache → DB
```

Spring example:

```
@Cacheable("patients")
public Patient getPatient(Long id) {
```

```
    return repository.findById(id).orElseThrow();
}
```

Recommended cache:

- Redis
- Caffeine
- Hazelcast

Cache reduces DB load dramatically.

---

4. Asynchronous Processing

Do not block request threads for slow operations like:

- sending emails
- generating reports
- calling external APIs

Use **@Async**.

Example:

```
@Async
public void sendEmail(Notification n) {
    emailService.send(n);
}
```

Flow:

```
HTTP Request
     |
     ├── Fast response
     |
     └── Background task
```

Thread pool configuration:

```
@Bean
Executor taskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(10);
```

```
        executor.setMaxPoolSize(50);
        executor.initialize();
        return executor;
    }
```

---

5. Use Messaging for Heavy Workloads

For **very high concurrency**, decouple services using queues.

Examples:

- RabbitMQ
- Kafka

Flow:

```
User Request
      |
      ▼
Spring API
      |
      ▼
Message Queue
      |
      ▼
Worker Service
```

Benefits:

- request returns fast
- work processed later
- prevents system overload

---

6. Use Non-Blocking IO (Reactive)

Traditional Spring MVC uses:

```
1 request = 1 thread
```

Reactive Spring WebFlux uses:

```
1 thread → thousands of requests
```

Architecture:

```
Event Loop
    |
    ├── Request1
    ├── Request2
    ├── Request3
```

Useful for:

- IO heavy APIs
- streaming
- gateway services

---

7. Database Optimization

Database is often the **main bottleneck**.

Key strategies:

Indexing

```
CREATE INDEX idx_patient_email ON patients(email);
```

Avoid N+1 queries

Bad:

```
1 query + N queries
```

Use:

```
JOIN FETCH
```

Pagination

Never load huge datasets.

```
Pageable pageable = PageRequest.of(0, 20);
repository.findAll(pageable);
```

---

8. Use Rate Limiting

Protect APIs from overload.

Example:

- Bucket4j
- API Gateway

Example rule:

```
100 requests per second per user
```

Flow:

```
Request
   |
Rate Limiter
   |
Allowed → Process
Rejected → 429 Too Many Requests
```

---

9. Horizontal Scaling

When one server is not enough.

Add multiple instances:

```
          Load Balancer
        /       |       \
    App1      App2      App3
```

Tools:

- Kubernetes
- AWS ELB
- NGINX

---

10. Use Circuit Breakers

Prevent cascading failures.

Example:

```
Service A → Service B → Service C
```

If Service C fails repeatedly:

```
Circuit opens
Requests fail fast
```

Tool:

- Resilience4j

---

11. Monitor Everything

Use:

- **Spring Boot Actuator**
- **Micrometer**
- **Prometheus**
- **Grafana**

Monitor:

```
CPU
Threads
DB pool
HTTP latency
Cache hit rate
```

Example metric:

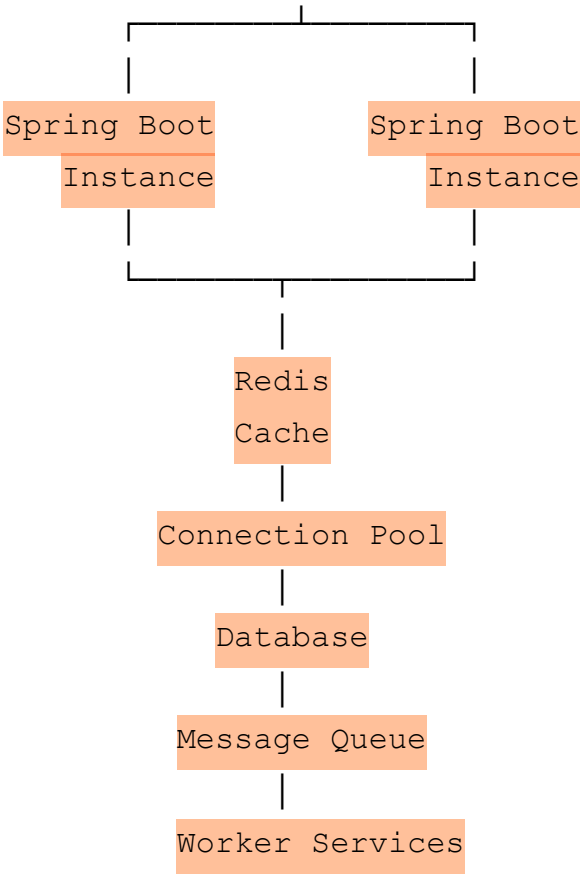
```
hikaricp.connections.active
```

---

12. Real Production Architecture

Typical high-scale Spring Boot system:

```
                Load Balancer
                      |
```



---

13. Common Bottlenecks in High Concurrency

Bottleneck	Solution
Thread pool exhaustion	Increase threads
DB connection limits	Use pooling
Slow queries	Indexing
Repeated queries	Cache
Blocking IO	Async
Heavy background work	Message queue

---

14. Golden Rule of Scalability

High concurrency systems follow this pattern:

```
Cache → Async → Queue → Scale Horizontally
```

✅ **In one sentence**

Handling high concurrency in Spring Boot requires optimizing **thread pools, connection pools, caching, asynchronous processing, messaging, and horizontal scaling**.

If you'd like, I can also show **how Netflix/large SaaS systems design high-concurrency Spring Boot architectures (with real numbers like 100k req/sec)** — it's very insightful for **system design interviews**.